



I2200/EE471: Digital Image processing

Lecture 1: Course Introduction and Prerequisites



Prof. YingLi Tian

Jan. 28, 2026

Department of Electrical Engineering
The City College of New York
The City University of New York (CUNY)

Some materials from Dr. Lexing Xie and Dr. Shahram Ebadollahi



Important for Graduate Study

- Ability of self-study
- Communication skills
 - Paper or report writing
 - Presentation skills
- Problem solving skills
- Academic writing skills
- Programming skills
- Work hard



Introduce yourself

- Name
- Research Direction
- Your Advisor if you have one
- Why take this course? Or your expectations
- Master or PhD
- University of your BA degree
- Home country
- Others you want to share: hobbies, ...



General Information

- **Class Time:** Wednesday 5:00pm – 7:45pm
- **Location:** Shepard S-276
- **TA:** Mr. Ryan Goldberg
- **Office Hours:**
 - Wednesday 11:00am-2:00pm In person (ST512)/ or by appointment to email the TA
 - Email: rgoldbe000@citymail.cuny.edu
- **Brightspace for communication**
 - Syllabus, assignments, and lecture slides, etc.



Pre-requisites

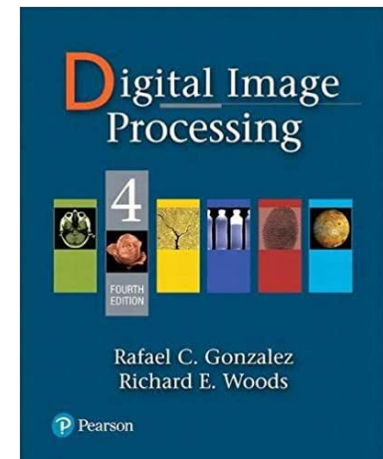
- For Undergraduate Students:
 - (Engr103 & EE30600) or CSC470
- For Graduate Students:
 - Signals & Systems
 - Linear Algebra
 - Probability
- All:
 - Matlab or other programming language



COURSE PROTOCOLS & POLICIES

Course textbook

- **Digital Image Processing, 4th Edition**
by Gonzalez and Woods, Pearson 2017
(ISBN-10: 9780133356724; ISBN-13: 978-0133356724)
- Very well written book
- Broad coverage of the subject
- Accessible by wide audience





Course Objective

- Learn fundamental technologies of digital image and video processing
- Use them in your research and daily life
- Improve your presentation skills
- Improve self-study skills
- Enhance research skills
- **Improve programming skills to solve a real problem**
- Have fun and get 3 credits!!



Course outline

- One mid-term exam
- 3 assignments
 - Practices – help to better understand what you have learnt. No need to hand in.
 - Homework – Finish before due day.
- One final project
 - Program
 - Report
 - Presentation in the class



Assignments

- 3 assignments
- Due time: Before midnight of the due day by email your homework to TA!
- One student will present the assignment in the next class after the HW due.
- Homework grades will be sent out the week after you hand in your assignments.
- Types of questions in the assignment:



Types of Questions in the Assignments

- Analytical
- Experimental
 - A problem which require some programming and experimentation. e.g.
 - 1) change parameters of an image processing algorithm, observe the effect, comment on your findings
 - 2) how do you think this image could be enhanced? Show it! Why this approach?



Late Policy for Assignments

- 1 day (1-24 hrs): 0.8
- 2-4 days (25-96 hrs): 0.5
- up to a week (7 days, or 148 hrs) late: 0.2
- after that 0.0
- Please save the returned email receipt due to communication problem.
- Ask an extension in advance if you have an emergency.



Grading Policy

- Homework 30%
- Mid Exam 30%
- Project report, program, and presentation: 40%

A+: 97~100; A: 93~96; A-: 90~92; B+: 87~89; B: 83~86; B-: 80~82; C+: 77~79; C: 73~76; C-: 70~72; D: 60~69; F: under 60



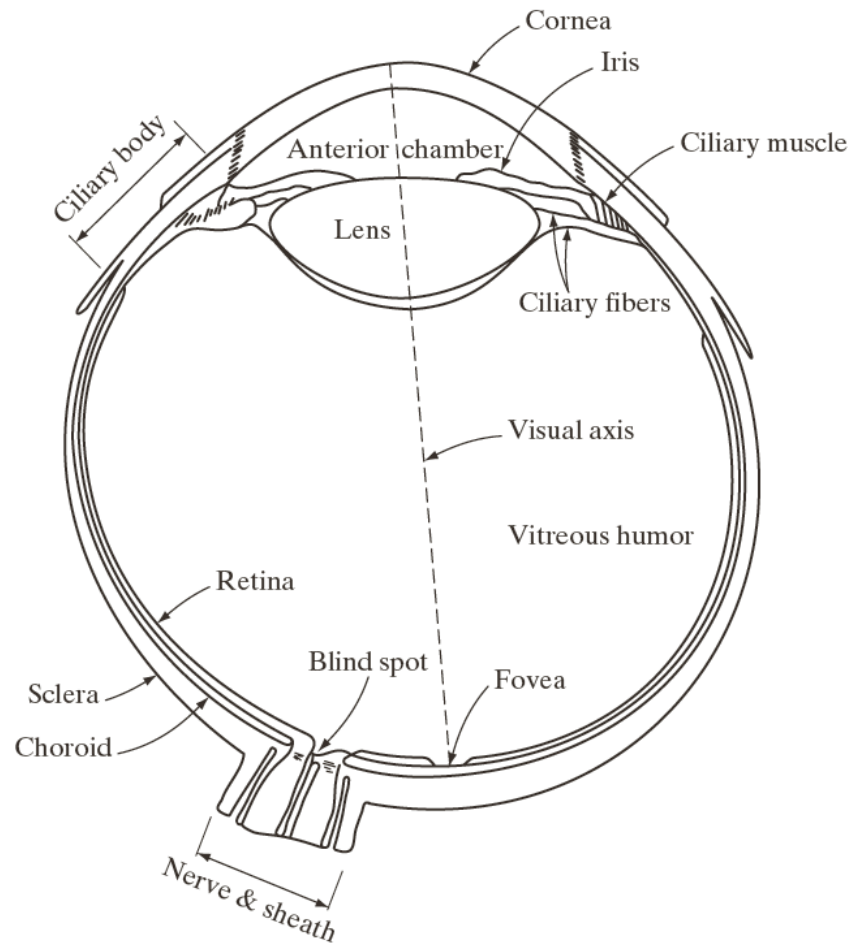
Class Policy

- Attend classes
- Be active in the class: ask questions, discussions
- **Zero-Tolerance on Cheating**
 - **Homework, Exam, & Final Project**
 - Anyone caught cheating will be dealt with according to applicable University policy.



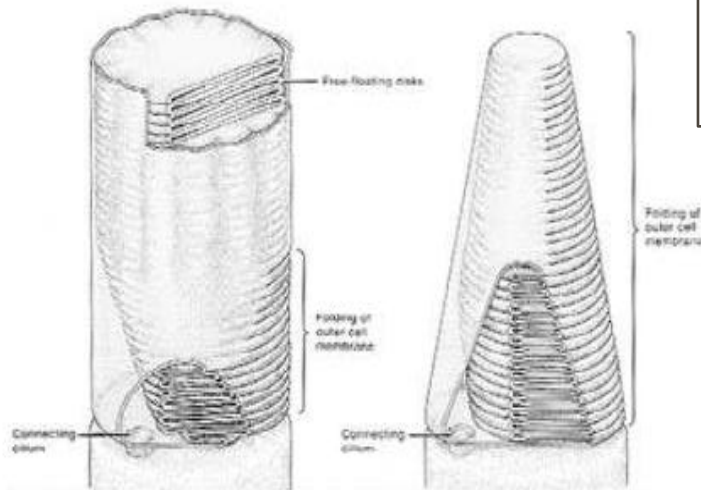
INTRODUCTION TO DIGITAL IMAGE PROCESSING

Eye Physiology



Simplified diagram of a cross section of the human eye

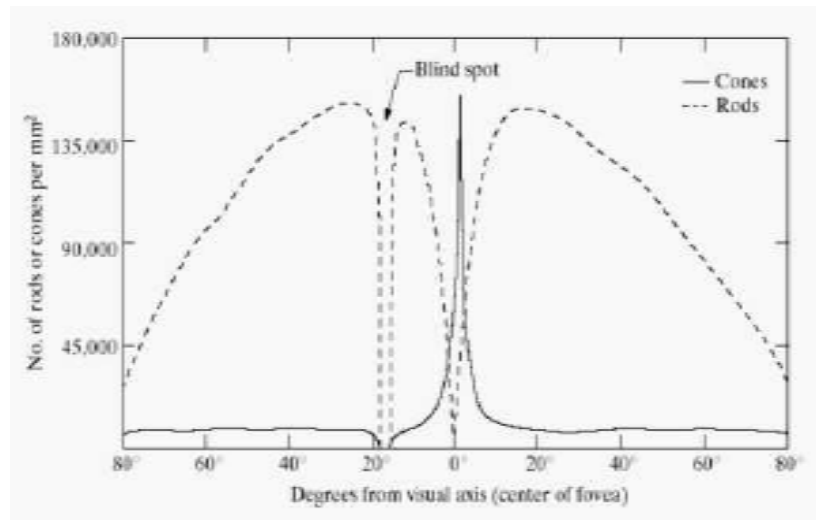
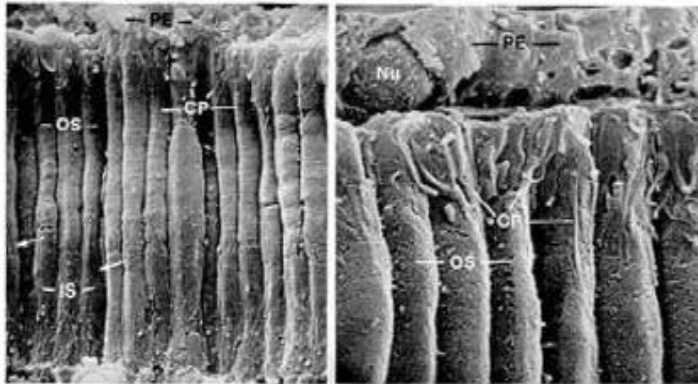
Eye Physiology & Visual Perception



- 75~150 Million
- Sensitive to low illumination
- Distributed over Retina
- Scotopic (dim light) vision

- 6~7 Million
- Highly sensitive to color
- Concentrated in Fovea
- Photopic (day light) vision

Rods & Cones Distribution in Retina



Photoreceptor Cells

Electromagnetic Spectrum & more

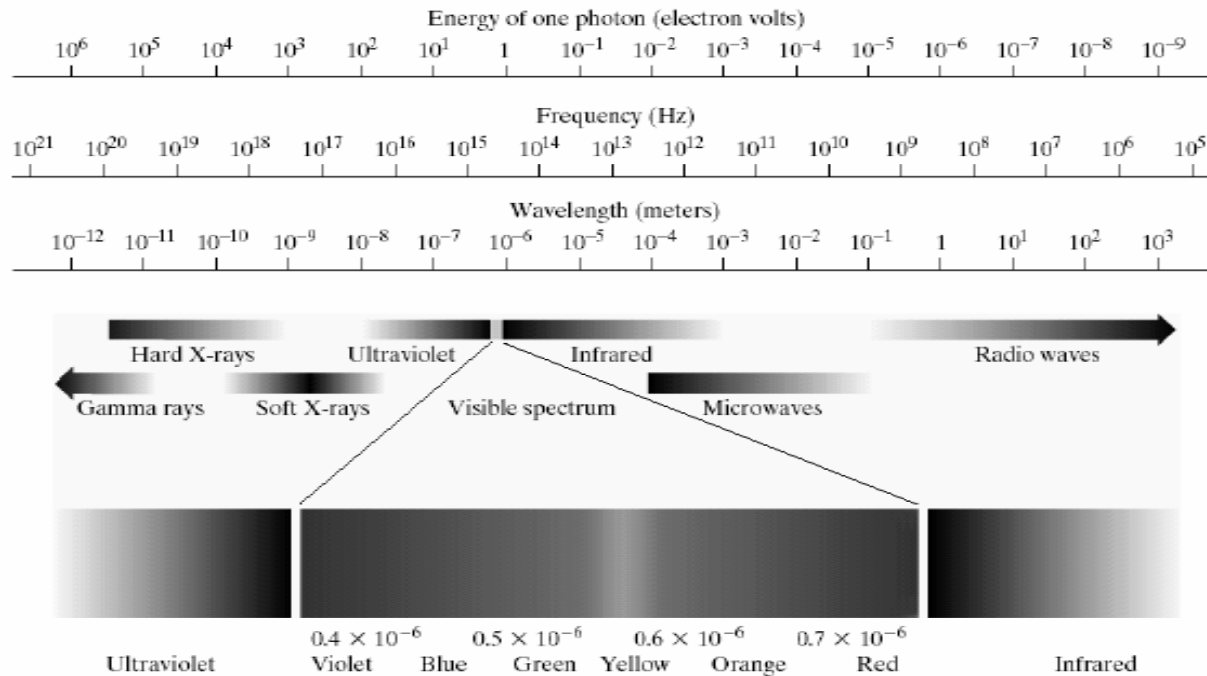
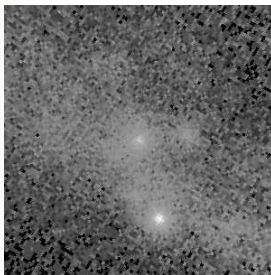
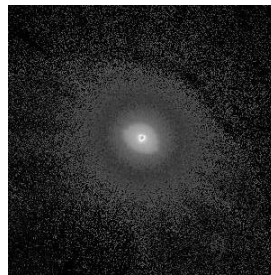


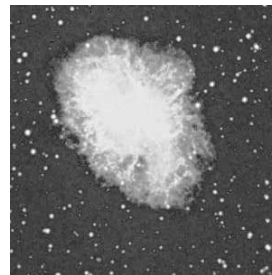
FIGURE 2.10 The electromagnetic spectrum. The visible spectrum is shown zoomed to facilitate explanation, but note that the visible spectrum is a rather narrow portion of the EM spectrum.



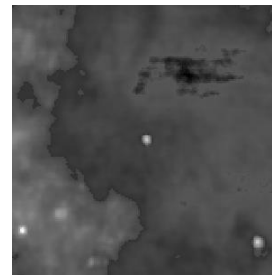
Gamma



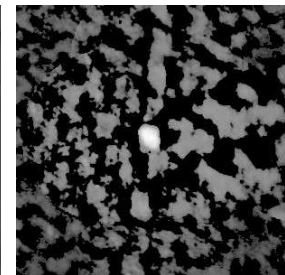
X-ray



Optical

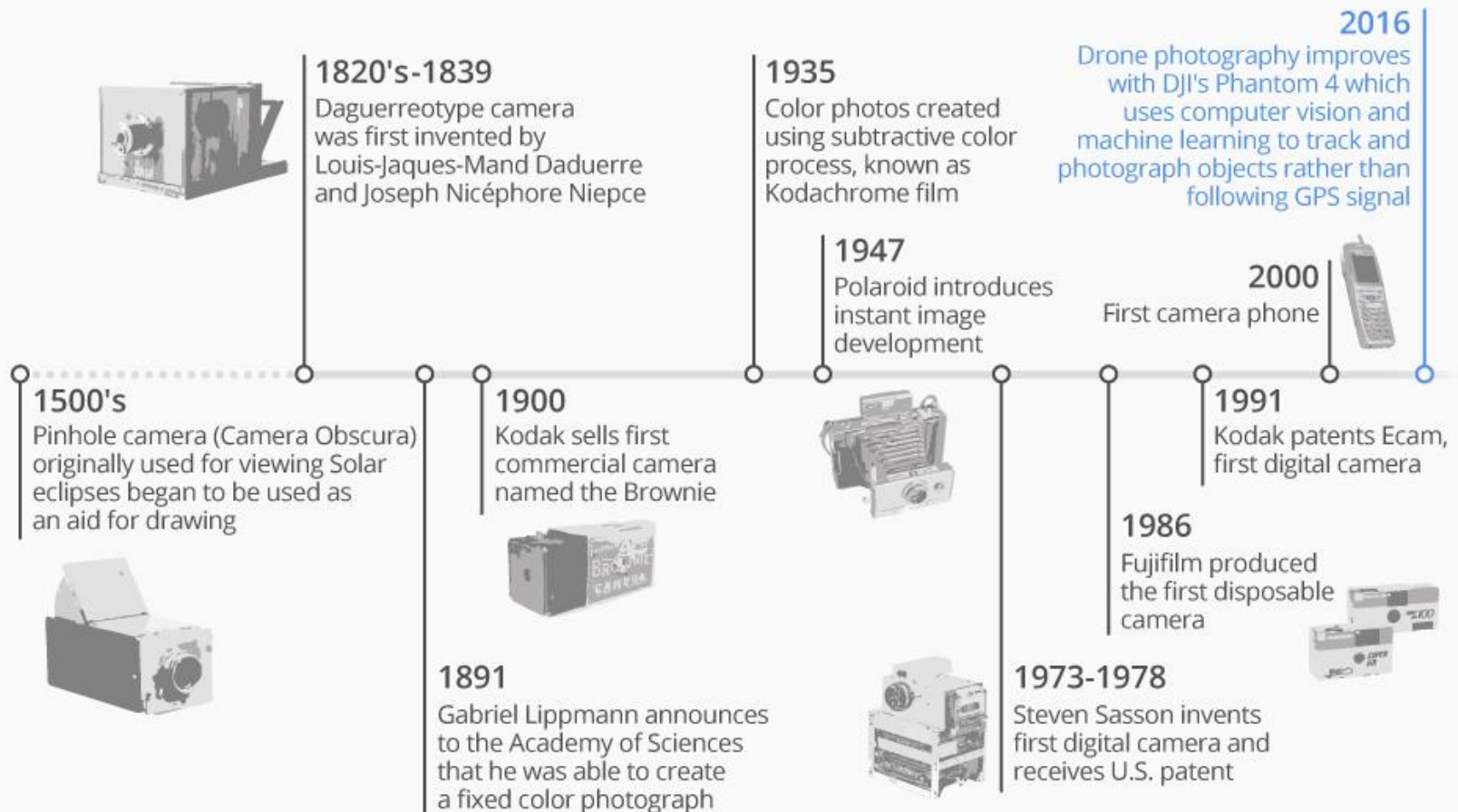


Infrared



Radio

Picture Perfect: The Evolution of the Camera



@StatistaCharts Source: Encyclopedia Britannica

statista

Image -- 1

■ Image = 2D function

$$f(x, y) = i(x, y) \cdot r(x, y)$$

$i(x, y)$ – illumination

$r(x, y)$ -- reflectance

$$0 \leq f(x, y) \leq F \text{ -- } 255$$

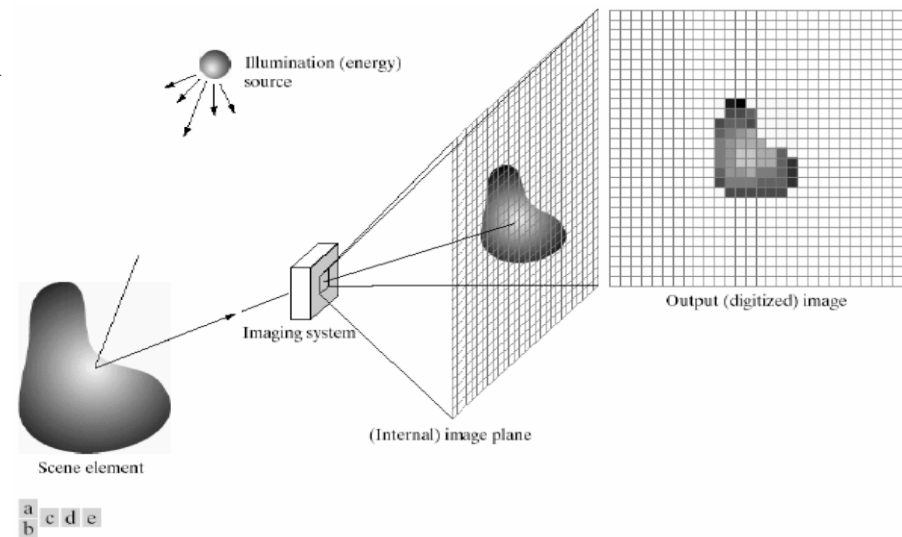


FIGURE 2.15 An example of the digital image acquisition process. (a) Energy ("illumination") source. (b) An element of a scene. (c) Imaging system. (d) Projection of the scene onto the image plane. (e) Digitized image.

Image -- 2

- Image = 2d function

$$f(x, y) = i(x, y) \cdot r(x, y)$$

$i(x, y)$ – illumination

$r(x, y)$ -- reflectance

$$0 \leq f(x, y) \leq F$$

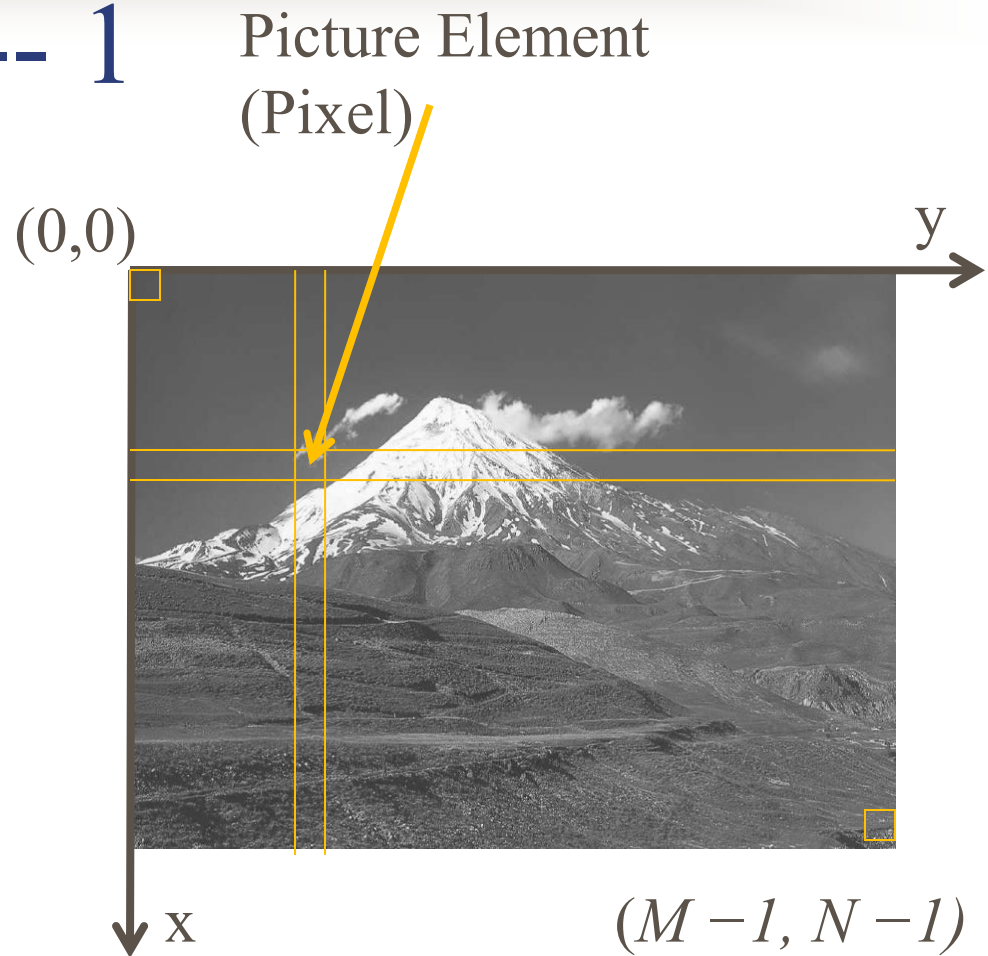
$$0 \leq x \leq X$$

$$0 \leq y \leq Y$$



Digital Image -- 1

- Discretization
 - Spatial sampling
 - Quantization

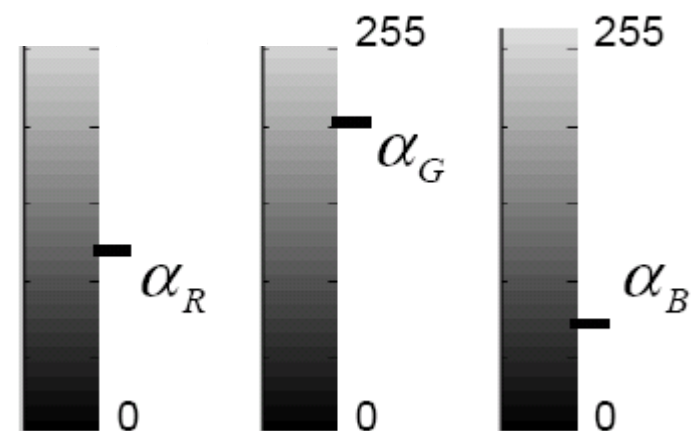
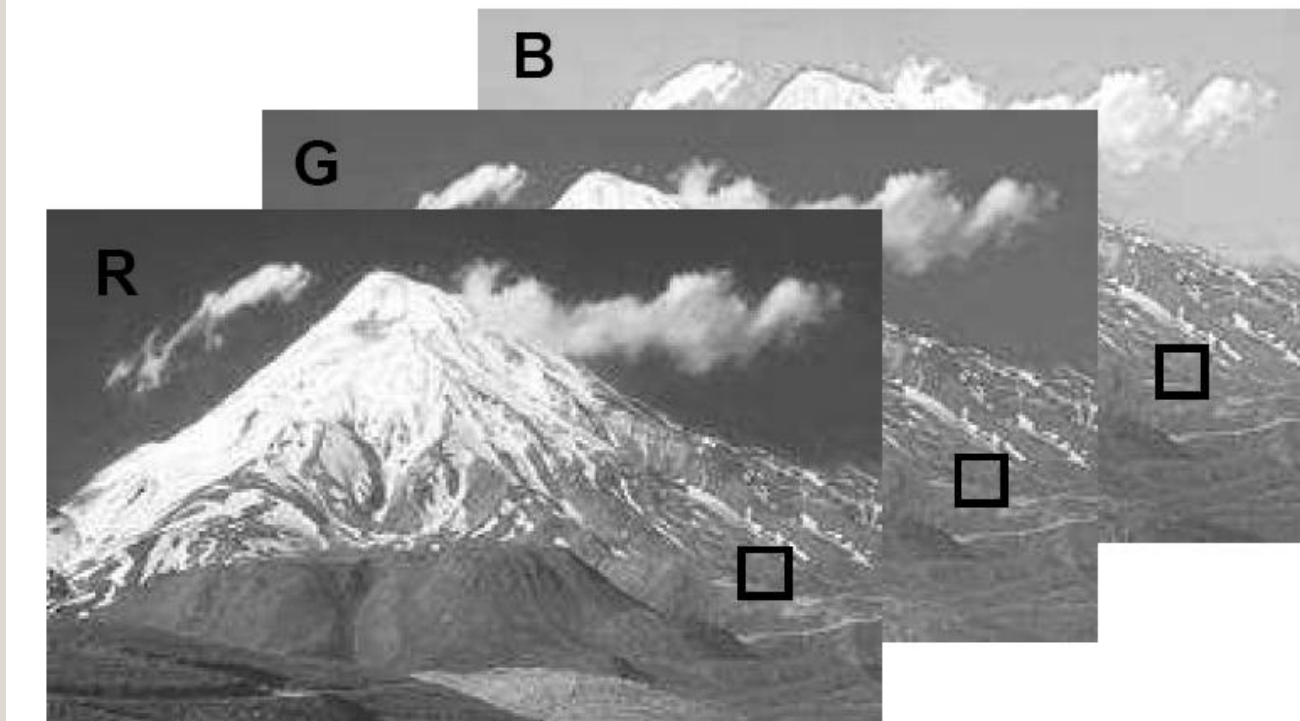


Digital Image -- 2

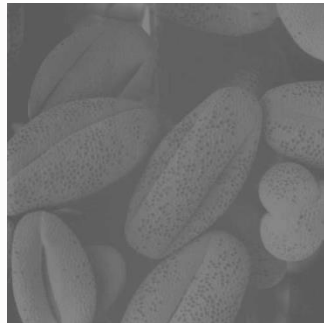
$$f = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \vdots & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$



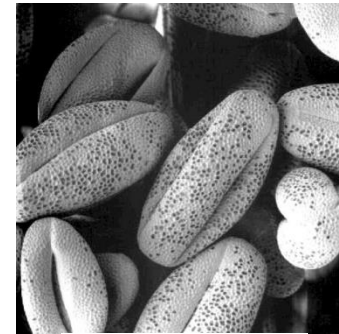
Color Image



Digital Image Processing



Low level



Mid level



$$\bar{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_P \end{bmatrix}$$

High level:







What are we going to study?

WEEK	TOPIC	BOOK CHAPTER
1	Course Introduction and Prerequisites	1
2	Digital image fundamentals	2
3	Gray-level and color image processing	6
4	Image enhancement in spatial domain	3
	Image Transform and Frequency	4
5	Domain Enhancement ((DFT, DCT, etc.)	
6	Image restoration	5.1-5.10
7	Morphological image processing	9
8	Image and Video compression and standards	8
9	Image segmentation	10
10	Image descriptors	11
11	Object recognition	12.1-12.2
12	Project presentation	



Why study Digital Image Processing?

- Image & video is a major communication media
 - “An image is worth 1000 words!”
- In all application domains, image and video is becoming indispensable
- WWW, faster computation, more storage, proliferation of image capture and consumption devices → Need for more, better, faster, and more intelligent image and video analysis
- It's fun!



Applications of Digital Image Processing?

- **Consumer domain**
 - Storage, tagging, searching,...
- **Security and Surveillance**
- **Remote sensing**
 - Agriculture, Urban growth monitoring, ...
- **Medical**
 - CAD, quantification, organization, assisted surgery, ...
- **Space explorations**
 - Image mosaic, image matching, ...
- **Art**
 - Working methods of painters, material used, ...
- **Military**
- **... and many more!**

Application Domains

- Consumer domain
 - Storage, tagging, searching,...

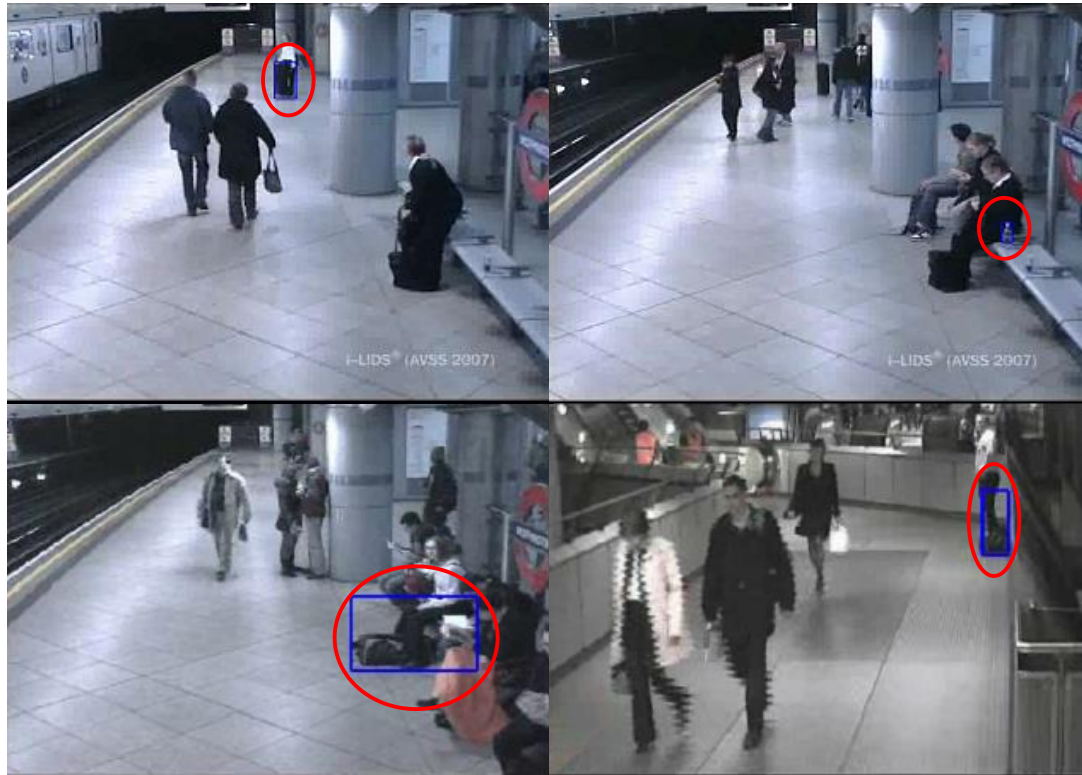


*mountain,
snow,
buildings,
sky*

Application Domains

■ Security and Surveillance

- Indoor, outdoor, traffic monitoring ...

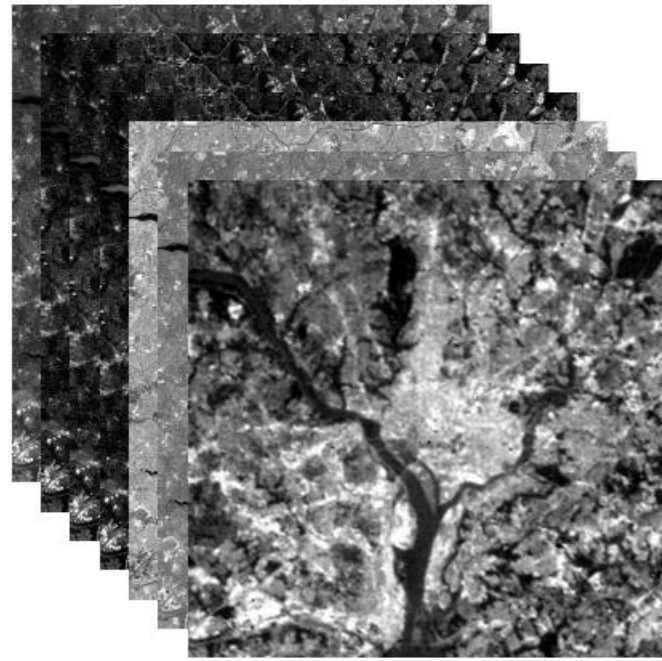


ABC7
News 1
IBM SSS

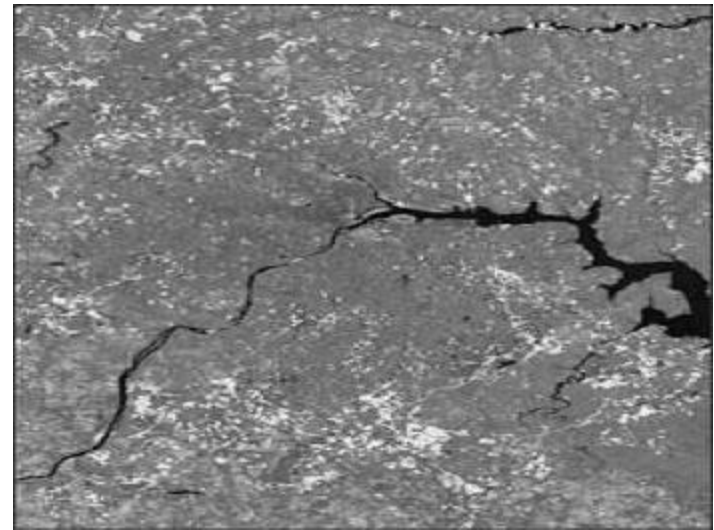
ABC7
News 2
IBM SSS

Application Domains

- Remote sensing
 - Agriculture, Urban growth monitoring, ...



Multi-spectral imaging



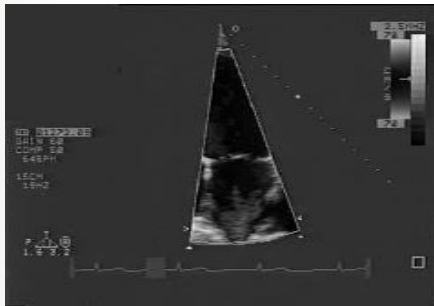
Urban growth in Washington D.C.
between 1973 and 1985

[Image courtesy of NASA/LANDSAT]

Application Domains

■ Medical

- CAD, quantification, organization, assisted surgery, ...



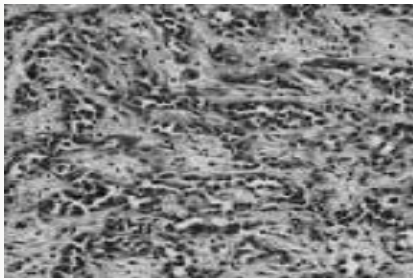
Color doppler Echocardiogram



Brain MRI



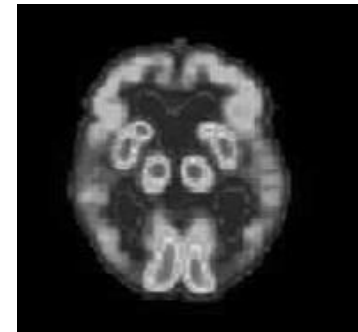
Cardiac CT



Tissue Microscopy



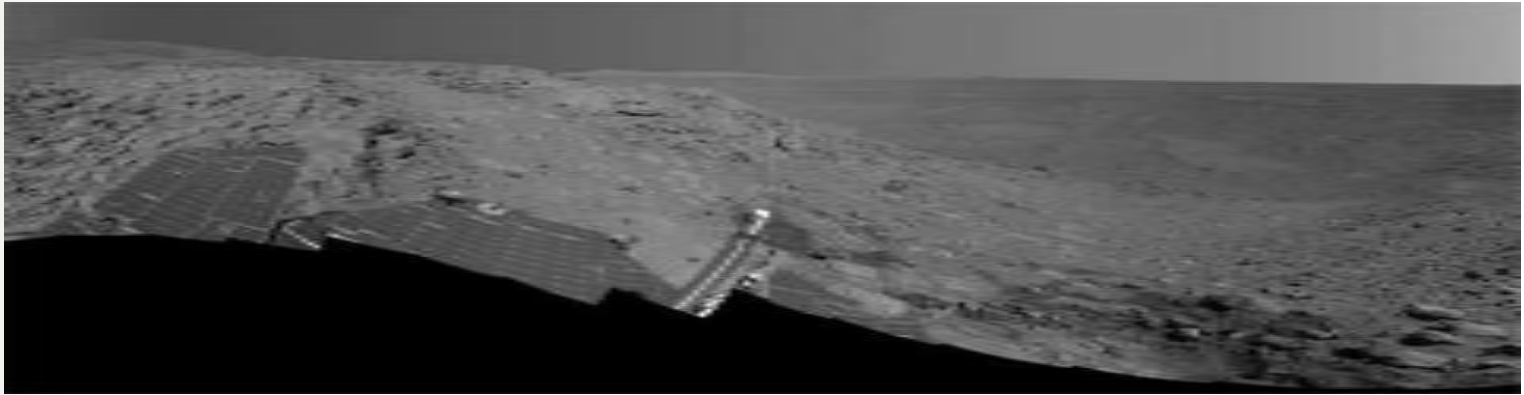
Spine X-ray



Brain PET (Alzheimers)

Application Domains

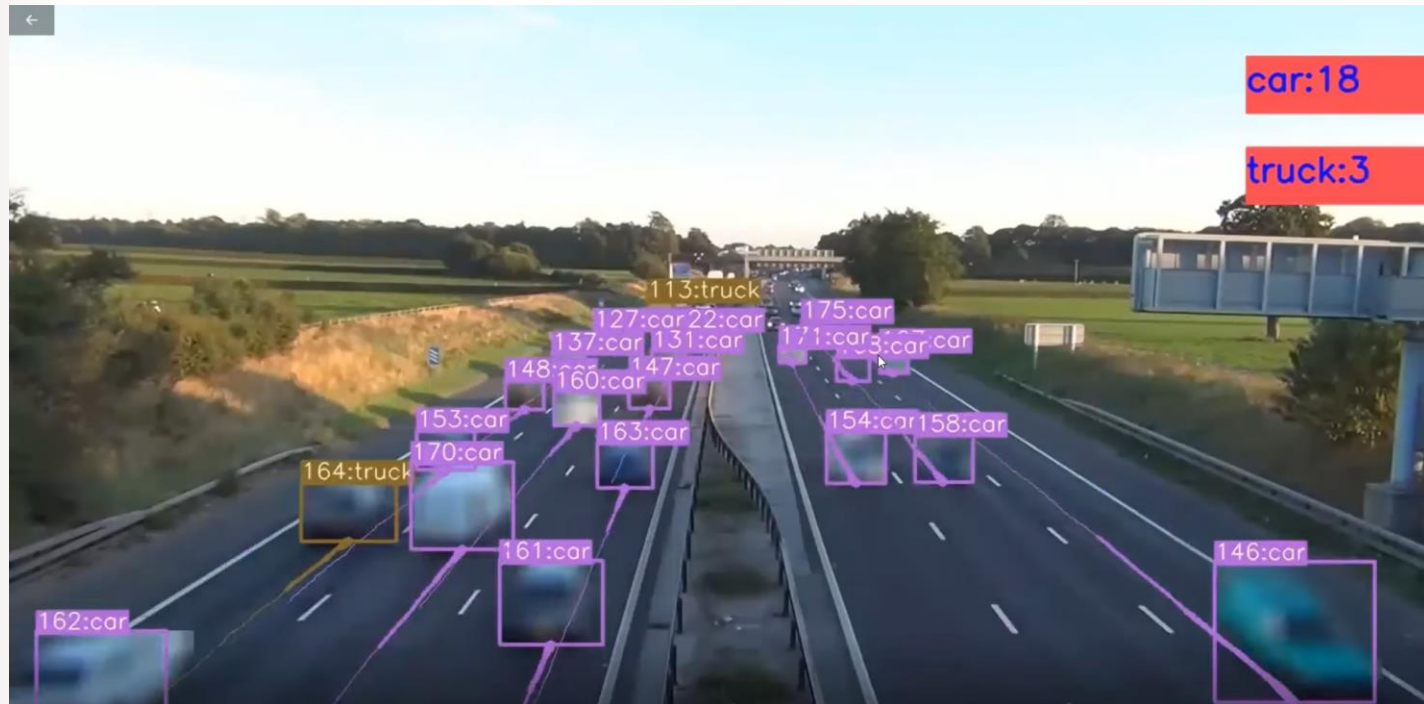
- Space explorations
 - Image mosaic, image matching, ...



Panoramic image built from images taken from mars rover Spirit
[image courtesy of NASA/JPL/Cornell]

Application Domains

- Scene Understanding
 - Object segmentation, detection, and recognition



<https://www.youtube.com/watch?v=QWrP77qXEMA>



TOOLS AND PREREQUISITES FOR IMAGE PROCESSING



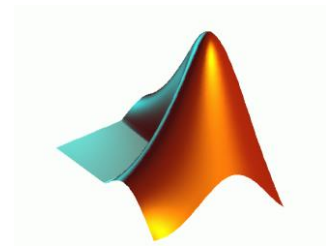
Outline

- Review and intro in MATLAB
 - A light-weight review of linear algebra and probability
 - An introduction to image processing toolbox
- A few demo applications
- Image formats in a nutshell
- Pointers to image processing software and programming packages

Matlab is ...

-  : a numerical computing environment and programming language. Created by The MathWorks, MATLAB allows easy matrix manipulation, plotting of functions and data, implementation of algorithms, creation of user interfaces, and interfacing with programs in other languages.

- Main Features:
 - basic data structure is matrix
 - optimized in speed and syntax for matrix computation





Why MATLAB?

- Shorter code, faster computation
- Focus on ideas, not implementation

$$f(x) = 2 \cdot \sin(x^3) / 3 + 4.56, x \in \{1, 3, 5, \dots, 9999\}$$

- C:

```
#include <math.h>
double x, f[500];
for( x=1.; x < 1000; x=x+2)
    f[(x-1)/2]=2*sin(pow(x,3.))/3+4.56;
```

- MATLAB:

```
f=2*sin((1:2:1000).^3)/3+4.56;
```

But: scripting language, interpreted,



Matrices

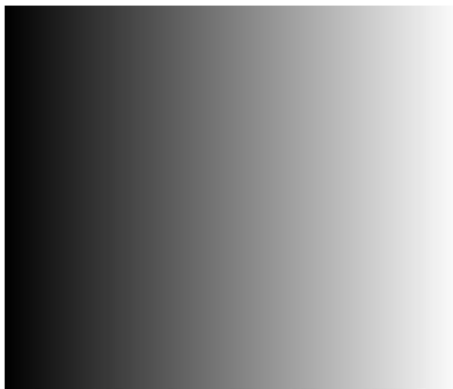
- ... are rectangular “tables” of entries where the entries are numbers or abstract quantities ...
- Some build-in matrix constructors
 - `a = rand(2)`, `b = ones(2)`, `c=eye(2)`,
- Addition and scalar product
 - `d = c*2`;
- Dot product, dot-multiply and matrix multiplication
 - `c(:)'*a(:)`, `d.*a`, `d*a`
- Matrix inverse, dot divide, etc.
 - `inv(a)`, `a./d`

Matrices as images, and vice versa

- `x = 1 : 256;`
- `y = ones(256,1);`
- `a = x*y;`
`b = y*x;`

`size(a) = ?` `size(b) = ?`

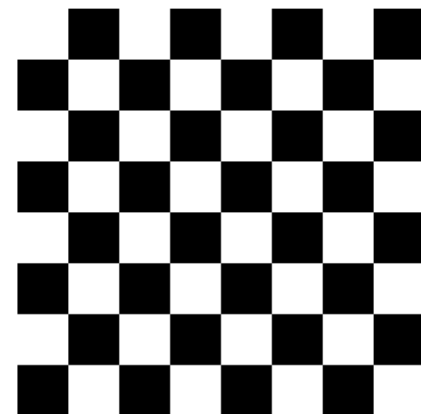
`imagesc(b); colormap(gray(256))`



`imagesc(checkerboard(32)>.5);`

or, from scratch:

```
b = ones(1,8); b(2:2:end)=0  
b = [b; b(end:-1:1)]  
b = repmat(b, [4 1])  
chessb = kron(b,ones(32));
```



256x256 chess board

Eigen vectors and eigen values

- “eigenvectors” are exceptional vectors in the same direction as Ax

$$Ax = \lambda x$$

- λ are called *eigenvalues*

- Examples:

- $A = \begin{bmatrix} .8 & .3 \\ .2 & .7 \end{bmatrix}$
- $[v, d] = \text{eig}(A);$
- $A * v(:, 1)$
- $A * v(:, 2)$

- eigshow

- properties of λ :

- $\sum_{i=1}^n a_{ii} = \sum_{i=1}^n \lambda_i = \text{trace}(A)$

- $\lambda_1 \cdot \lambda_2 \cdot \dots \cdot \lambda_n = \det(A)$

- eigen-vectors and values are useful for:

- Getting exponents of a matrix A^{100000}
- Image compression
- Object recognition
- The search algorithm behind Google
- ...

Probability

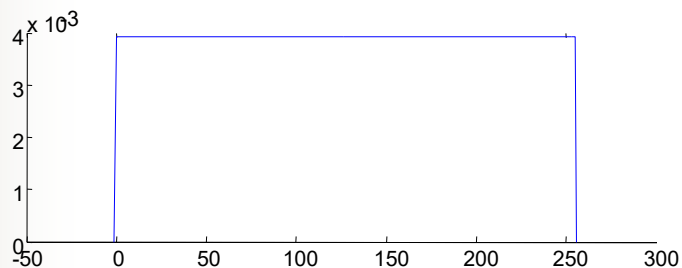
- *probability* refers to the *chance* that a particular event (or set of events) will occur.



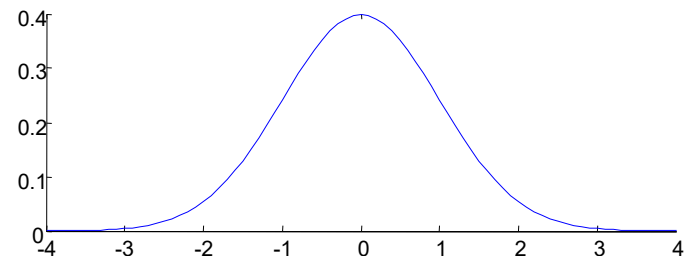
$$\Pr(\text{head})=1/2, \Pr(\text{tail})=1/2$$

- *probability density function* $p(x)$ is a non-negative integrable function $\mathbb{R} \rightarrow \mathbb{R}$ such that for any interval $[a, b]$: $\Pr(x \in [a, b]) = \int_a^b p(x)dx$

```
p = pdf('uniform', -1:256, 0, 255);  
plot(-1:256, p)
```

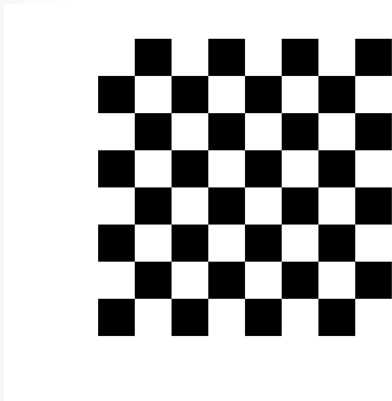


```
p = pdf('normal', -4:.1:4, 0, 1);  
plot(-4:.1:4, p)
```



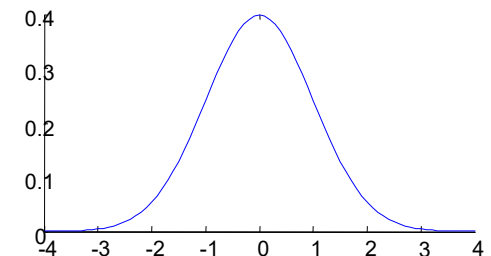
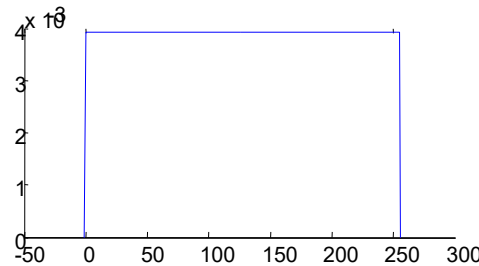
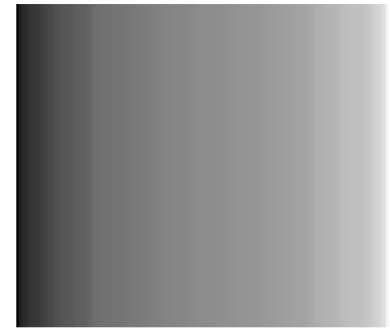
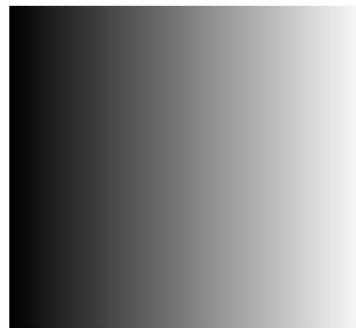
Probability

- Suppose you're blind-folded and points to a point in a cardboard with the following prints, after a friend rotates and shifts it randomly (i.e. randomly draw a pixel from the following images)



$$p(\blacksquare) = 1/2$$

$$p(\square) = 1/2$$



$$p(\blacksquare) = p(\blacksquare) = \dots = p(\square) = 1/256$$

Mean and std

■ Mean

■ $m_x = E[x] = \int x p(x) dx$

■ Standard-deviation

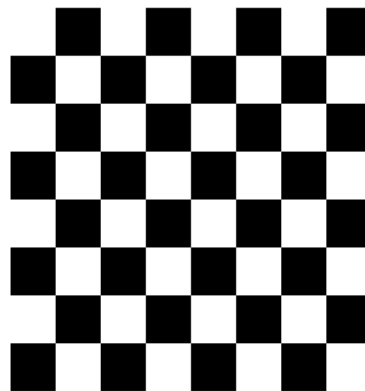
■ $\sigma_x^2 = E[(x-m_x)^2] = \int (x-m_x)^2 p(x) dx$

(a) and (b) are aforementioned gray-scale images with values between $[0,1]$. Which one of the following holds, if any?

$m_a < m_b$ **X** $m_a = m_b$

$\sigma_a < \sigma_b$ **X** $\sigma_a > \sigma_b$

(a)



(b)





MATLAB (contd.)

- M-files:
 - functions
 - scripts
- Language constructs
 - Comment: %
 - if .. else... for... while... end
- Help:
 - help function_name, helpwin, helpdesk
 - lookfor, demo



MatLab Image ToolBox



What Is Image Processing Toolbox?

Image Processing Toolbox is a collection of functions that extend the capability of the MATLAB numeric computing environment. The toolbox supports a wide range of image processing operations, including

- Spatial image transformations
- Morphological operations
- Neighborhood and block operations
- Linear filtering and filter design
- Transforms
- Image analysis and enhancement
- Image registration
- Deblurring
- Region of interest operations



Images in MATLAB and Image Processing Toolbox

MATLAB stores most images as two-dimensional arrays (i.e., matrices), in which each element of the matrix corresponds to a single pixel in the displayed image. (Pixel is derived from picture element and usually denotes a single dot on a computer display.)

For example, an image in resolution of 200x300 composed of 200 rows and 300 columns of different colored dots would be stored in MATLAB as a 200-by-300 matrix. Some images, such as true color images, require a three-dimensional array, where the first plane in the third dimension represents the red pixel intensities, the second plane represents the green pixel intensities, and the third plane represents the blue pixel intensities.

Pixel Coordinates

In Matlab, the image is treated as a grid of discrete elements, ordered from top to bottom and left to right, as illustrated by the figure.

For pixel coordinates, the first component r (the row) increases downward, while the second component c (the column) increases to the right. Pixel coordinates are integer values and range between 1 and the length of the row or column.

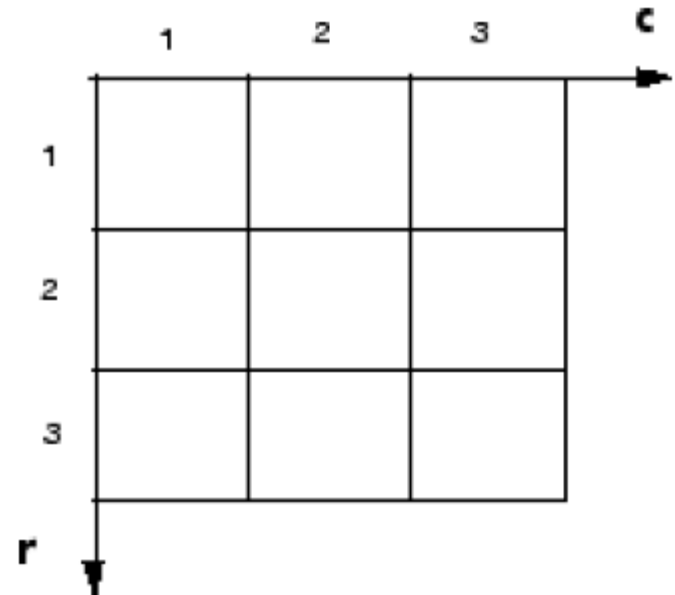




Image Types in the Toolbox -- 1

Image Type	Interpretation
Binary (Also known as a <i>bilevel</i> image)	Logical array containing only 0s and 1s, interpreted as black and white, respectively. See “Binary Images” on page 2-8 for more information.
Indexed (Also known as a <i>pseudocolor</i> image)	Array of class <code>logical</code>, <code>uint8</code>, <code>uint16</code>, <code>single</code>, or <code>double</code> whose pixel values are direct indices into a colormap. The colormap is an m-by-3 array of class <code>double</code>. For <code>single</code> or <code>double</code> arrays, integer values range from $[1, p]$. For <code>logical</code>, <code>uint8</code>, or <code>uint16</code> arrays, values range from $[0, p-1]$. See “Indexed Images” on page 2-9 for more information.



Image Types in the Toolbox -- 2

Image Type	Interpretation
Grayscale (Also known as an <i>intensity</i> , <i>gray scale</i> , or <i>gray level</i> image)	Array of class uint8, uint16, int16, single, or double whose pixel values specify intensity values. For single or double arrays, values range from [0, 1]. For uint8, values range from [0,255]. For uint16, values range from [0, 65535]. For int16, values range from [-32768, 32767]. See “Grayscale Images” on page 2-11 for more information.
Truecolor (Also known as an <i>RGB</i> image)	<i>m</i>-by-<i>n</i>-by-3 array of class uint8, uint16, single, or double whose pixel values specify intensity values. For single or double arrays, values range from [0, 1]. For uint8, values range from [0, 255]. For uint16, values range from [0, 65535]. See “Truecolor Images” on page 2-12 for more information.



Read an image from an image file -- 1

Clear the MATLAB workspace of any variables and close open figure windows.

```
>> close all
```

To read an image, use the *imread('image_filename')* command. *A = imread(filename, fmt)* reads a grayscale or color image from the file specified by the string filename.

Example:

```
>> I = imread('pillsetc.png');
```

imread function can read different format of images include TIFF, GIF, PNG, etc.

If the file is not in the current directory, or in a directory on the MATLAB path, specify the full pathname.



Read an image from an image file -- 2

Example:

```
Image1 = imread('E:\pillsetc.png');
```

Or change the “current directory” of MATLAB



Display an image

Use function of **imshow(I)**.

Example:

```
>> close all
```

```
>> clear all
```

```
>> I = imread('pillsetc.png');
```

```
>> whos % See "I" in Matlab
```

Name	Size	Bytes	Class	Attributes
------	------	-------	-------	------------

I	471x661x3	933993	uint8	
---	-----------	--------	-------	--

```
>> figure, imshow(I);
```



Write an image to an image file -- 1

The *imwrite* function writes an image to a graphics file in one of the supported formats. The most basic syntax for `imwrite` takes the image variable name and a filename. If you include an extension in the filename, MATLAB infers the desired file format from it.

Example:

```
>> imwrite(I, 'mytest.jpg');  
>> imfinfo(I);
```



Write an image to an image file – 2

```
>> imfinfo('mytest.jpg')
```

```
ans =
```

```
    Filename: 'mytest.jpg'
```

```
    FileModDate: '14-Nov-2008 10:42:32'
```

```
    FileSize: 45735
```

```
    Format: 'jpg'
```

```
    FormatVersion: ''
```

```
    Width: 661
```

```
    Height: 471
```

```
    BitDepth: 24
```

```
    ColorType: 'truecolor'
```

```
    FormatSignature: ''
```

```
    NumberOfSamples: 3
```

```
    CodingMethod: 'Huffman'
```

```
    CodingProcess: 'Sequential'
```

```
    Comment: {}
```

Project 1: Improve Image Contrast -- 1

Read the image “pout.png” and display it. Then improve image contrast and write the improved image to a file.

Step 1: Read and Display an Image

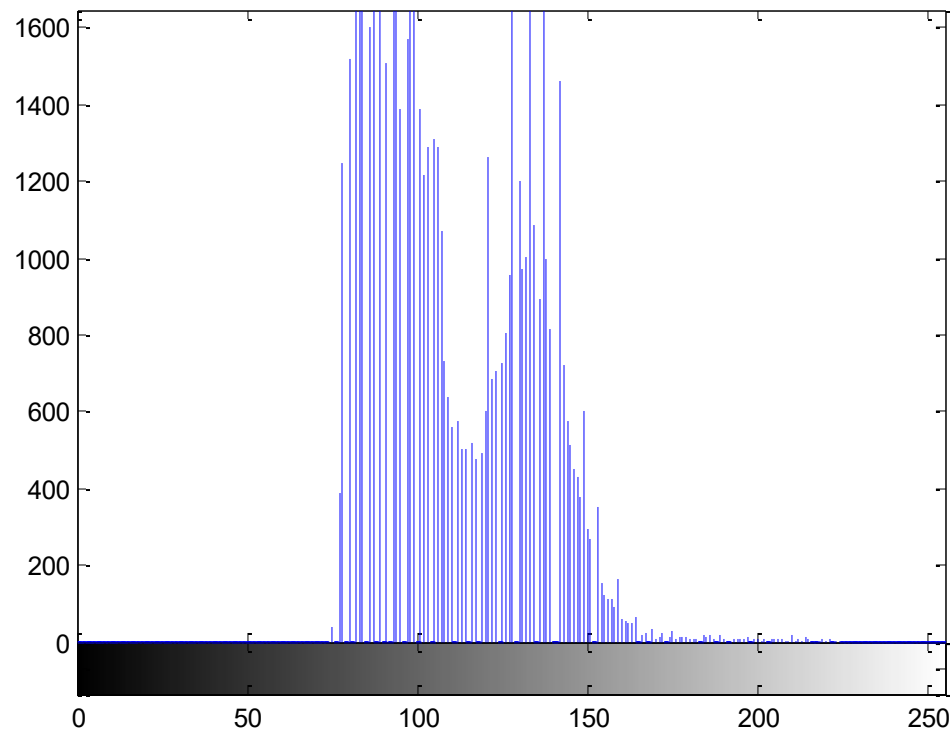
```
>> close all  
>> clear all  
>> I = imread('pout_grey.png');  
>> imshow(I)
```



Project 1: Improve Image Contrast -- 2

Step 2: Check the distribution of image intensities

```
>>figure, imhist(I)
```



Project 1: Improve Image Contrast --3

Step 3: Improve Image Contrast by the *histeq* function to spread the intensity values over the full range of the image, a process called *histogram equalization*.

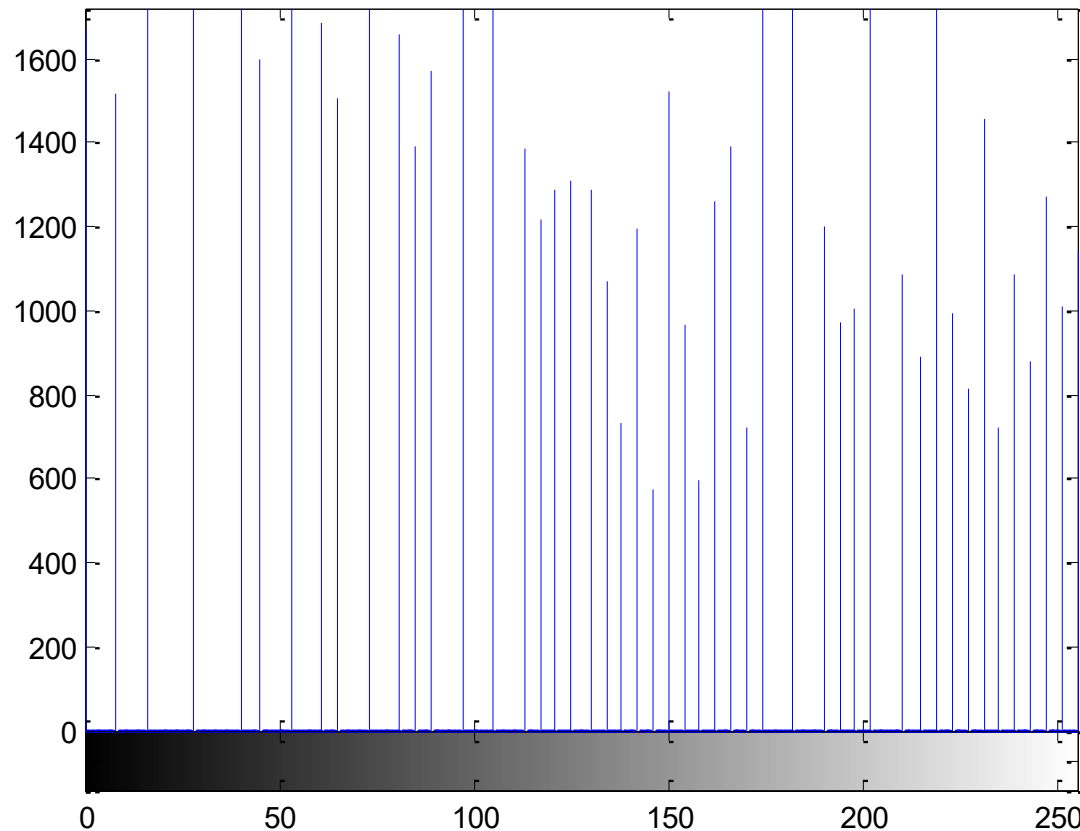
```
>> I2 = histeq(I);
```

```
>> figure, imshow(I2) % Display I2 in a new figure window.
```



Project 1: Improve Image Contrast -- 4

Step 4: Check the distribution of equalized image intensities
>>figure, imhist(I2)





Project 1: Improve Image Contrast -- 5

Step 5: Write the Image to a file and check the contents of the newly written file

```
>>imwrite (I2, 'pout2.png');
```

```
>>imfinfo('pout2.png')
```

```
ans =
```

```
    Filename: 'pout2.png'
```

```
    FileModDate: '29-Dec-2005 09:34:39'
```

```
    FileSize: 36938
```

```
    Format: 'png'
```

```
    FormatVersion: []
```

```
    Width: 240
```

```
    Height: 291
```

```
    BitDepth: 8
```

```
    ColorType: 'grayscale'
```

```
    ...
```

Project 2: Edge Detection -- 1

Goal: Give an image lenna_grey.jpg, detect the edges of the image

%Step 1: Read Image

```
>> close all; clear all;
```

```
>> I= imread(' lenna_grey.jpg ');
```

```
>> imshow(I);
```

%Step 2: Get Edge Image by
Different Method

```
>> BW1 = edge(I, 'sobel');
```

```
>> BW2 = edge(I,'prewitt');
```

```
>> BW3 = edge(I,'roberts');
```

```
>> BW4 = edge(I,'canny');
```



Project 2: Edge Detection -- 2

```
>> figure; imshow(BW1);  
>> figure; imshow(BW2);  
>> figure; imshow(BW3);  
>> figure; imshow(BW4);
```



Sobel Edge



Prewitt Edge



Roberts Edge



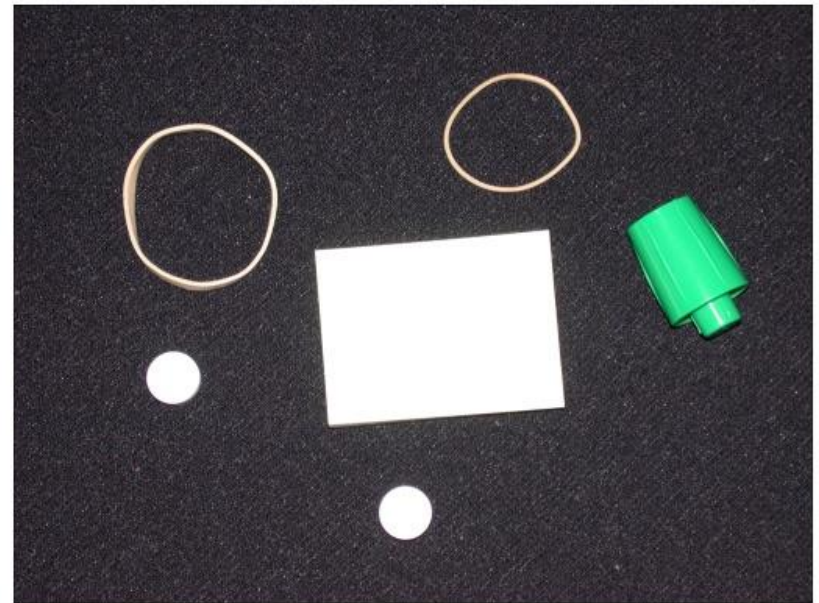
Canny Edge

Project 3: Find Boundaries of Objects

-- 1

Goal: Give an image `pillsetc.png`, classify objects based on their roundness using `bwboundaries()`, a boundary tracing routine.

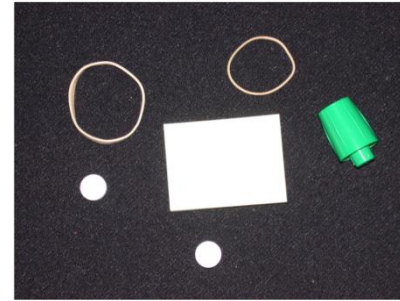
- Step 1: Read Image
- Step 2: Threshold the Image
- Step 3: Remove the Noise
- Step 4: Find the Boundaries



Project 3: Find Boundaries of Objects

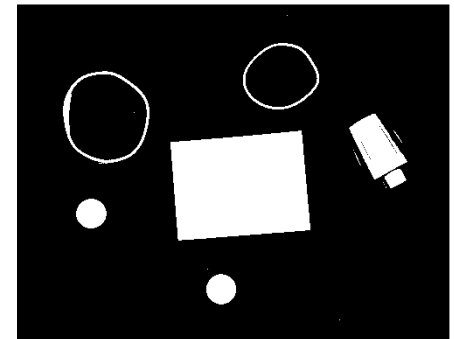
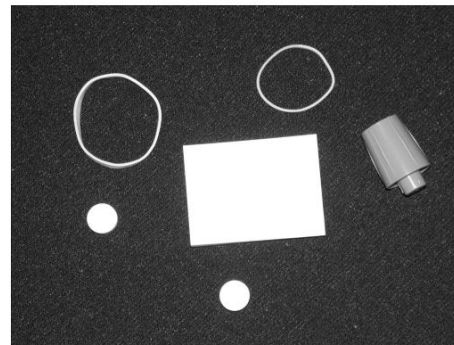
-- 2

```
%Step 1: read in pillsetc.png.  
>> close all; clear all;  
>> RGB = imread('pillsetc.png');  
>> imshow(RGB);
```



```
%Step 2: Threshold the Image, Convert the image to black  
and white in order to prepare for boundary tracing using  
bwboundaries.
```

```
>> I = rgb2gray(RGB);  
>> imshow(I);  
>> threshold = graythresh(I);  
>> bw = im2bw(I, threshold);  
>> imshow(bw);
```



Project 3: Find Boundaries of Objects

-- 3

%Step 3: remove the noise, Using morphology functions, remove pixels which do not belong to the objects of interest.

% remove all object containing fewer than 30 pixels

```
>> bw = bwareaopen(bw,30);
```

```
>> imshow(bw)
```

% fill a gap in the pen's cap

```
>> se = strel('disk',2);
```

```
>> bw = imclose(bw, se);
```

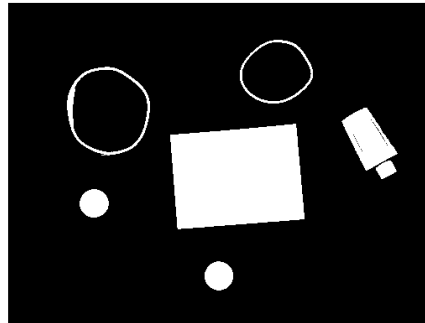
```
>> imshow(bw)
```

% fill any holes, so that regionprops can be used to estimate

% the area enclosed by each of the boundaries

```
>> bw = imfill(bw,'holes');
```

```
>> imshow(bw)
```





Project 3: Find Boundaries of Objects

-- 4

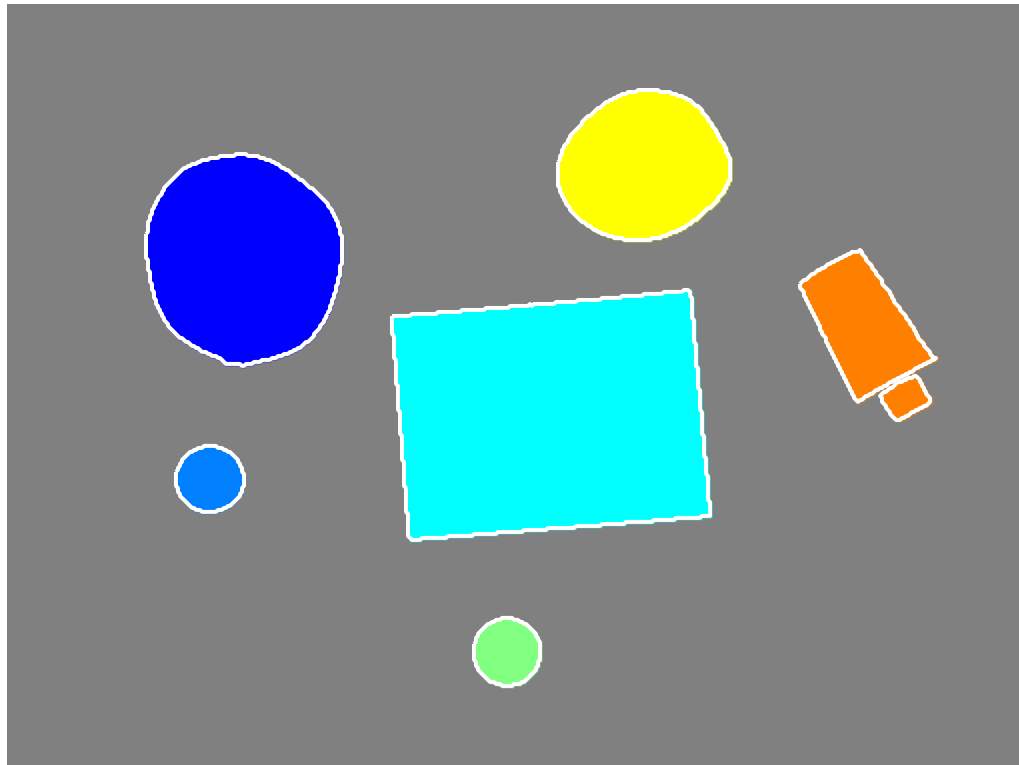
%Step 4: Find the boundaries, concentrate only on the exterior boundaries. Option 'noholes' will accelerate the processing by preventing bwboundaries from searching for inner contours.

```
>>[B,L] = bwboundaries(bw,'noholes');  
% Display the label matrix and draw each boundary  
>> imshow(label2rgb(L, @jet, [.5 .5 .5]))  
>> hold on  
>> for k = 1:length(B)  
    >> boundary = B{k};  
    >> plot(boundary(:,2), boundary(:,1), 'w', 'LineWidth', 2)  
>> end
```

Project 3: Find Boundaries of Objects

-- 5

%Step 4: Find the Boundaries





For more functions and examples

Go to MATLAB “Help → Image Processing Toolbox”

Have fun!!!



Introduction and pointers to other image processing software and programming packages

Demo of image processing software

- Enhancement
“equalize”
- Compression
- Color manipulation

with **GIMP** www.gimp.org

before



after



- unshake before



after





Image Processing Software

- Bitmap editing: Adobe Photoshop, Macromedia Fireworks
- Vector graphics editing: Adobe Illustrator, Corel Draw
- Consumer photo tools: Picassa, ACDSee, Windows Paint, XV, Photoshop Elements ...
- **GIMP or Adobe Photoshop**



Video processing software

- Player

- Windows media player, Real, Quicktime, iTunes, intervideo WinDVD, ...

- Format conversion

- ffmpeg

- Editing

- Adobe premier, muvee,



Image Processing Toolboxes

- In C/C++
 - OpenCV <https://opencv.org/>
- In Matlab: Image Processing Toolbox
<https://www.mathworks.com/products/image.html>
- In Java
 - Java Media APIs: JAI, JMF, Java image I/O ...
https://docs.oracle.com/javase/8/docs/technotes/guides/imageio/spec/imageio_guideTOC.fm.html
- In Python
 - Python Imaging Library (PIL)
<https://web.archive.org/web/20201121102218/http://www.pythonware.com/products/pil/>



Image Data Types

- Basic unit in disk: byte (8 bits)
- Images are stored as unsigned integers (0-255)
- Depends on the color space and the precision / bit depth
 - 1bit, 4bit, 8bit, 24bit, 32bit (+alpha channel), indexed colors (gif, 2-8 bits)
- In MATLAB:
 - uint8 → double → uint8

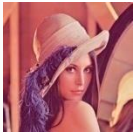


File Formats

- Why different file formats?
 - Convenient to use
 - Compact representation
- How many formats do we have?
 - e.g. 30+ in a consumer image software (ACDSee)
 - There are much more out there:
raster, vector, metafile, ... and growing
- Basic structure: Header + Data

Format Comparison

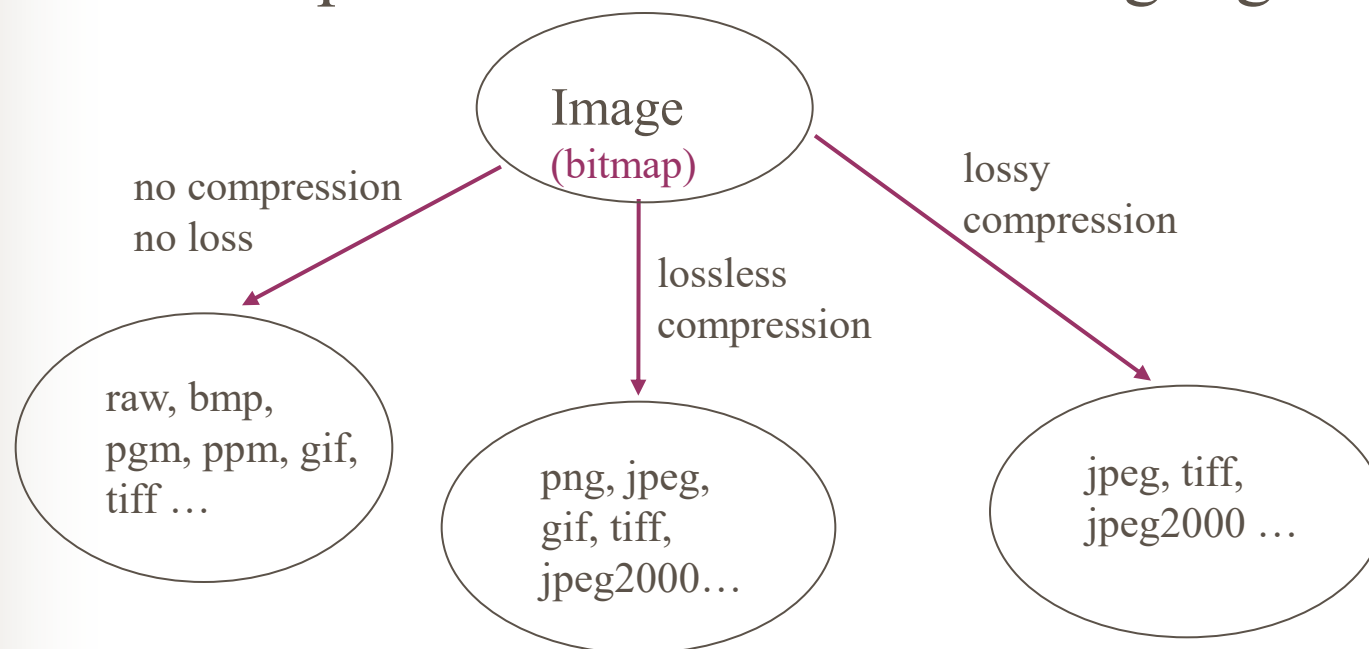
Two 256x256 color images

Format	RAW	BMP	GIF	PNG	JPG
Lossy?	N	N	N	N	Y
Compressed?	N	N	Y	Y	Y
	192K	193K	52.2K	106K	16K
	192K	193K	5K (4bit)	23K	20K
Fine prints	Raw data	Header ~1K	Look-up table + data		Quality factor 80

Why do the two images have different sizes as GIF/PNG/JPG files ?

Image Format Classification

- Types that MATLAB supports:
 - BMP, JPEG, PNG, GIF, TIFF, XWD, HDF, PCX, ...
 - Other open-source libraries from “google”



Resources and pointers



- Google, Wikipedia, Mathworld ...
- Getting Help in Matlab
 - Matlab help, Image Processing Demos
 - DIP matlab tutorial online
 - Usenet groups



Announcement

- Next class
 - Sampling & Quantization